ARCHITECTURE PATTERN REFERENCE · NATS-TECH-LAB

Event Sourcing + CQRS + NATS

A design-pattern deck for the decisions that shape an event-sourced logistics platform — each card states the intent, the fork, the forces, and the POC's verdict.

THE QUESTION EVERY CARD SERVES

What is the correct responsibility split between **JetStream** (event backbone), **NATS KV** (fast lookup / watch / cache), **Postgres** (transactional store), and **CQRS projections**?

PATTERN FAMILIES

- Modeling — is it even event-sourced?
- Projection — shaping the read side
- Safety — making the log trustworthy
- Consistency — where truth & invariants live
- Identity — what an aggregate is keyed by
- Performance — taming replay cost

Companion to the Obsidian "Event Sourcing" deep-dive

7 decisions · 2 cross-cutting concerns

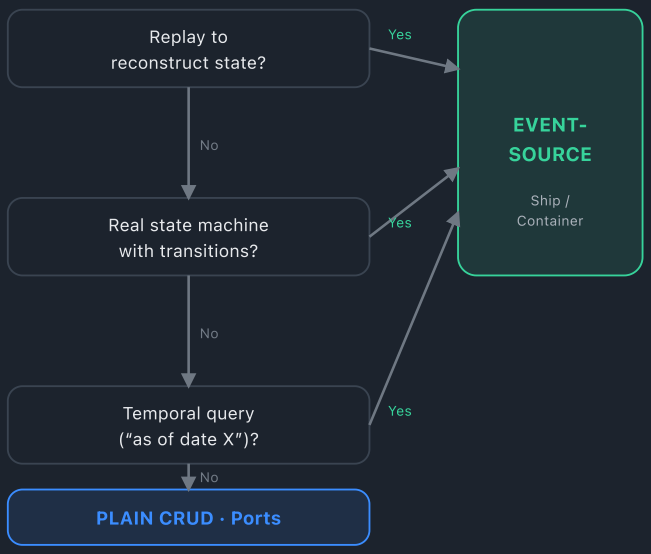
01 MODELING

Event-source it, or plain CRUD?

Not everything in a domain earns an event stream. Forcing one onto data that doesn't need it adds subjects, consumers and projections for no benefit.

DECISION Does anything need to **replay this to reconstruct state**? — not "does it change over time?"

DECISION FLOW



WORKED CONTRAST

	SHIP / CONTAINER	PORT
Write	cmd → rule → event	direct row write
State machine	yes	none
Replayed?	Shape C rebuilds	never
Audit	transitions *are* the fact	`created_at`

Trap: "is it reference data" isn't the whole test — a rate table ("in effect on X") secretly needs history.

POC VERDICT **Ship & Container** are event-sourced; the **Ports registry** is plain Postgres CRUD — no lifecycle worth replaying.

Phase 9.5 / 9.6 ✓

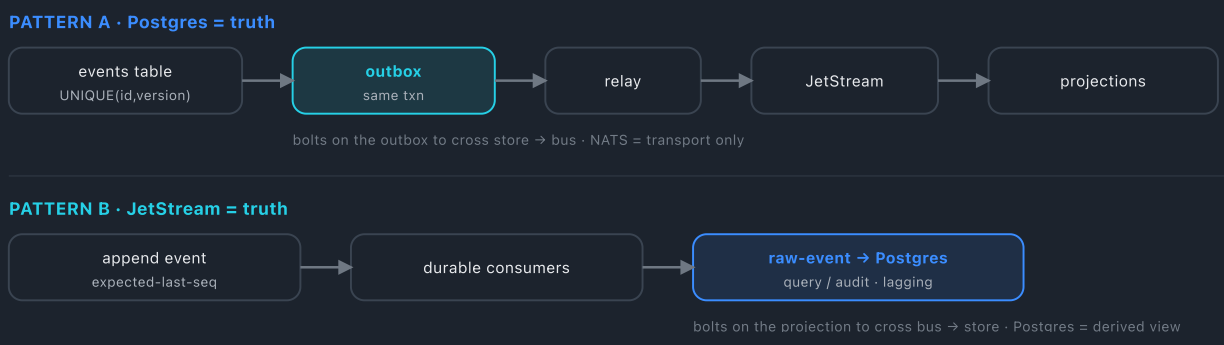
02 CONSISTENCY

Which system owns the truth?

Two coherent patterns — and they are mirror images. Each bolts on exactly the component the other gets for free.

DECISION Is the durable source of truth the **Postgres event store** (A) or the **JetStream stream** (B)?

THE SAME BRIDGE, BUILT FROM OPPOSITE BANKS



FORCES

	A · POSTGRES	B · JETSTREAM
Free	SQL-queryable truth	native distribution
Bolt-on	outbox	raw projection
Cross-agg txn	yes	saga
Global order	natural	per-subject

WHY IT MATTERS HERE

The real difference is **where transactionally-consistent, queryable history lives** — the source itself (A) or a lagging derivative (B).

Only **B actually tests JetStream as an event store**; A would relegate NATS to transport, dodging the very question the POC exists to answer.

POC VERDICT

Pattern B — JetStream is the source of truth; Postgres & KV are downstream projections, never written by the command path. Its footguns (Cards 06–07) are deliberately load-bearing.

Working assumption ✓

03 PROJECTION

What shape is the read model?

The POC builds all three shapes side-by-side so the trade-off is felt, not argued.

DECISION Read from a **KV projection**, a **cache in front of Postgres**, or **reconstruct from the log** with no stored model at all?

SHAPE A · KV

1 event → projector → KV

2 read = KV lookup

No Postgres. Cheapest reads; KV *is* the read model.

SHAPE B · KV + PG

1 event → Postgres (canonical)

2 write-through KV

3 read: KV hit → return; miss → PG → warm KV

SHAPE C · REPLAY

1 read → replay `seq=1`

2 fold aggregates

No stored model. The defining ES property: state from history alone.

TRADE-OFFS

SHAPE	STORE	COST AT SCALE
A	KV bucket	cheap reads
B	PG + KV	cheap + fallback
C	none	grows with depth → snapshots (Card ⌘)

KV HAS TWO ROLES — DON'T CONFUSE THEM

Write-side snapshot — safe for hard rules *if* replayed to the latest seq.

Read-model projection — eventually consistent; soft checks / display only. Never validate a hard rule against it.

POC VERDICT

All three, side-by-side. A & B show event-driven CQRS into persistent projections; C proves reconstruction — clear the stores, hit the endpoint, state still returns.

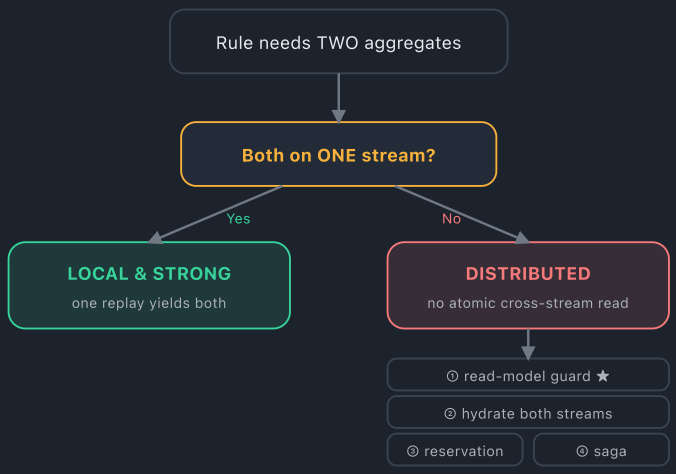
04 CONSISTENCY

Cross-aggregate invariants

A rule needing two aggregates' state (e.g. BR-008: don't load a container whose destination equals the ship's current port). The difficulty is the aggregate boundary, not the stream topology.

DECISION Are both aggregates on **one stream** (one atomic replay) or **two** (no atomic cross-stream read)?

THE FORK



OPTIONS & CONSISTENCY

OPTION	CONSISTENCY
① Read-model guard ★ cheap; stale-read window	eventual
② Hydrate both contexts coupled	**strong**
③ Reservation store extra write-side store	**strong**
④ Saga / compensate most moving parts; "correct" DDD	eventual, self-correcting

POC VERDICT **One stream first** (Phase 8) keeps invariants strong from a single replay. Phase 12 splits containers into TERMINAL — one variable changed — then defaults to the **read-model guard**.

Phase 8 ✓ · 12 ◔

05 IDENTITY

Aggregate identity — surrogate or natural key?

When the human-facing natural key can be wrong or must change, don't make it the aggregate's identity.

DECISION Fold the aggregate by an immutable **surrogate UUID**, or by the **natural key** (needing an alias-aware corrective event when it changes)?

SURROGATE KEY (UUID)

- **Folded by** immutable UUID at creation
- **Alias layer** needed on every natural-key read/write — permanent
- **Retrofit cost** high — routes/keys/fold all move
- **Use when** the natural key is an external interchange standard you may need to correct

NATURAL KEY + CORRECTIVE EVENT

- **Folded by** the business ID (alias-aware on correction)
- **Alias layer** only when a correction happens — rare
- **Retrofit cost** low — one new event type
- **Use when** already keyed by natural ID everywhere; corrections are one-off

THE POC SPLIT

	CONTAINER	SHIP
Natural key	ISO 6346 (`TCKU…`) — external standard	internal slug (`my-ship`)
Correction risk	**real — the `TCKU0001` fix**	**none forces it**
Identity	**surrogate UUID**; container ID = mutable attr	natural key stays identity

POC VERDICT **Container → surrogate UUID** (its key is an ISO standard it doesn't control). **Ship → natural key** — a surrogate there would be indirection without benefit. | Phase 8.3 ✓

06

SAFETY · PRODUCER SIDE

Write-side safety

Making JetStream trustworthy *as a store*, not merely as a log. Two producer-side gaps stand between the two.

DECISION How do you stop concurrent commands and client retries from corrupting the source of truth?

GAP · LOST INVARIANTS

Two concurrent commands hydrate the same pre-state, both validate, both publish — jointly violating a rule (same container on two ships).

FIX · OPTIMISTIC CONCURRENCY

Publish with `Nats-Expected-Last-Subject-Sequence`; on reject, re-hydrate → re-validate → retry (bounded).

GAP · RETRY DOUBLE-WRITE

An HTTP client retrying after a timeout durably appends the business event twice — and in event-store mode the duplicate *is* the record.

FIX · PUBLISH DEDUP

`Nats-Msg-Id` from a command idempotency key; configure the stream `Duplicates` window explicitly.

WHY THE SUBJECT TAXONOMY IS LOAD-BEARING HERE

Per-aggregate concurrency only works because the subject carries the `{id}` token — `expected-last-subject-sequence` is scoped per subject. With events split across `…{id}.arrived` / `…{id}.departed`, verify the `-Subject` variant, or fall back to one subject per aggregate.

POC VERDICT

Planned as **Phase 10** — optimistic concurrency + publish dedup, kept transport-agnostic in the `Publisher` port so `jetstream` types don't leak into the application layer.

Phase 10 ○

07

SAFETY · CONSUMER SIDE

Projection hardening

Safe under redelivery and reordering by engineering, not by accident. Today's safety rests on unexamined consumer defaults.

DECISION How does a projector guarantee a stale or duplicated redelivery can't clobber newer state?

GUARDED WRITE (CAS LOOP)

```
# value carries the source event seq
stored.seq = 9281
if event.seq <= stored.seq: skip
else: KV.Update(key, val,
        expectRevision) # CAS

# Postgres mirror
ON CONFLICT … WHERE
  excluded.seq > current.seq
```

ALSO DECIDED, NOT DEFAULTED

- **Ordering** — verify `Consume()` concurrency & set `MaxAckPending` explicitly
- **Retention** — `LimitsPolicy` with "unbounded is deliberate" documented; truth must never erode
- **Poison messages** — dead-letter subject or documented ack-and-log, never a silent drop

EVERY PROJECTOR, BY CONSTRUCTION

Idempotent
re-applying an event never corrupts the model

Ordered where required
same-entity events applied in sequence

Checkpointed
tracks last processed stream position

Version-aware
upcasters for old event schemas

POC VERDICT

Planned as **Phase 11** — sequence-guarded CAS writes replace naive `Put`; explicit limits & poison-message policy make the "never discard" property a decision, not an absence.

Phase 11 ○

✦ Snapshots — taming replay cost

PERFORMANCE

Both write-side hydration (`hydrate()`) and Shape C (`ReconstructFleet`) replay the log **from seq=1 on every call**. Latency grows linearly with stream depth — so for any long-lived aggregate, snapshots are a **when, not an if**.

THE REHYDRATION FLOW

Load snapshot state @ lastStreamSequence ▶ **Replay only after** events past that seq ▶ **Rebuild + validate** ▶ **Append**

A SNAPSHOT MUST CARRY ITS POSITION

```
{
  "aggregateId": "ship.SH-001",
  "state": { … },
  "lastStreamSequence": 9281
}
```

…or you can't know which events still need replaying after it.

WHERE IT LIVES · HOW FAR TO TRUST IT

Where	write-side **snapshot KV** / PG table — **not** a read-model projection
Trust	authoritative for hard rules *because* replayed to latest seq
Trigger	every N events / scheduled; always rebuildable from the log

POC ANGLE · PHASE 13 MEASURES THE CURVE

Shape C becomes unusable beyond a few thousand events without snapshotting; a busy ship's `hydrate()` slows its own writes. A snapshot is a **write-side optimization (rehydration)** — same event-folding code as a projection, different purpose and trust level.

⚠ Things to be aware of

PITFALLS

SOURCE-OF-TRUTH CAVEATS · PATTERN B

- **Retention must never discard.** If messages age out, truth **silently erodes** — the most dangerous default.
- **The Postgres copy lags.** Never read it on the write path or treat it as authoritative.
- **You debug a lagging copy.** The SQL you inspect may trail the stream by a few events.

THE DECIDING CAPABILITY

Concurrency control is what separates a store from a log: Postgres `UNIQUE` · stream `expected-seq` · Redis = hand-rolled → keep it off the write side.

Exactly-once doesn't exist
at-least-once + dedupe on id/seq → **every projection idempotent**

Per-subject ordering only
no global timeline across subjects → **aggregate-scoped projections**

No cross-aggregate atomic write (B)
invariants → **saga / reservation**, not one txn (Card 04)

Dual-write (A)
outbox or CDC — never "write DB then publish"

Correcting bad history
corrective event — never delete+republish (destroys audit, doesn't scale)

◆ POC map — where each decision is exercised

9.5/9.6	Event-source vs CRUD · Ship/Container vs Ports table	DONE
–	Source of truth = JetStream (Pattern B) · locked working assumption	LOCKED
1/2/6	Read-model shapes A / B / C · side-by-side panels	DONE
8	Cross-aggregate — one stream · strong from single replay	DONE
8.3	Surrogate key · Container UUID identity	DONE
9	Subject taxonomy · {region}.events.{tenant}.{agg}.{id}.{event}	DONE
10	Write-side safety · optimistic concurrency + dedup	PLANNED
11	Projection hardening · sequence-guarded CAS writes	PLANNED
12	Stream split + guard · TERMINAL stream, read-model guard	PLANNED
13	Load / degradation curves · k6: Shape C replay, hydration	PLANNED
14	Server-enforced tenancy · NATS accounts spike	OPTIONAL